

MIPS32 Assignment 1

Greatest Common Divisor (GCD) of two numbers

Contents

1	Introduction	2
2	Assignment 1: Greatest Common Divisor (GCD)	3
2.1	Objective	3
2.2	Background Concepts Needed	3
2.3	Task Description	4
2.3.1	GCD of positive numbers	4
2.3.2	GCD of negative numbers	4
2.4	Requirements	4
2.5	Sample Run	4

1 Introduction

This assignment build directly on the starter program you already have (MIPS_example_code.S), which reads numbers from the user with `syscall 5`, prints messages with `syscall 4`, and uses comparison branches (`bgt`, `blt`) with labels and an unconditional branch (`b`) to decide which code path to take. It reads three numbers and prints the sum of the two largest.

```
1 .data #data segment
2 prompt1: #prompt user to enter the numbers
3     .ascii "Enter the first number\n"
4 prompt2:
5     .ascii "Enter the second number\n"
6 prompt3:
7     .ascii "Enter the third number\n"
8 output: #output message
9     .ascii "The sum of the largest two of the three numbers : "
10 newline: #newline
11     .ascii "\n"
12 .text #code starts here
13 .globl main
14 main:
15
16     la $a0,prompt1 #printing the message for entering first number
17     li $v0,4 # which operation or service the OS should perform...
18     syscall
19
20     li $v0,5 #reading the first number and storing it in register $t0
21     syscall
22     move $t0,$v0
23
24     la $a0,prompt2 #printing the message for entering second number
25     li $v0,4
26     syscall
27
28     li $v0,5 #reading the second number and storing it in register $t1
29     syscall
30     move $t1,$v0
31
32     la $a0,prompt3 #printing the message for entering third number
33     li $v0,4
34     syscall
35
36     li $v0,5 #reading the third number and storing it in register $t2
37     syscall
38     move $t2,$v0
39
40     bgt $t0,$t1,test_1_2 #checking for the greater number between first and second number
41     blt $t0,$t1,test_0_2
42
43 test_1_2:
44     bgt $t1,$t2,sum01
45     blt $t1,$t2,sum02
46
47 test_0_2:
```

```

48  bgt $t0,$t2,sum01
49  blt $t0,$t2,sum12
50
51  sum01:
52      add $t3,$t0,$t1
53      b print_sum
54
55  sum02:
56      add $t3,$t0,$t2
57      b print_sum
58
59  sum12:
60      add $t3,$t1,$t2
61      b print_sum
62
63  print_sum:
64      la $a0,output
65      li $v0,4
66      syscall
67
68      move $a0,$t3
69      li $v0,1
70      syscall
71
72      la $a0,newline
73      li $v0,4
74      syscall
75
76      li $v0,10 #exit
77      syscall

```

Listing 1: Provided starter code (MIPS_example_code.S)

2 Assignment 1: Greatest Common Divisor (GCD)

2.1 Objective

Practice turning a comparison (like the `bgt/blt` pattern in the starter code) into a **loop**, and use it to compute the Greatest Common Divisor (GCD or HCF) of two positive integers.

2.2 Background Concepts Needed

- Everything already used in the starter code: `la`, `li`, `move`, `syscall`, `bgt`, `blt`, `b`, labels.
- New idea: a **loop**, made by branching back to an earlier label instead of only branching forward.
- `bltz` ("branch if less than zero") checks the sign bit if the number is negative.
- The **subtraction-based Euclidean algorithm**:
 - While $a \neq b$: if $a > b$, set $a = a - b$; otherwise set $b = b - a$.
 - When $a = b$, that value is the HCF.

2.3 Task Description

2.3.1 GCD of positive numbers

Write a MIPS32 program that:

1. Asks the user to enter two positive integers (reuse the prompt/read pattern from the starter code).
2. Repeatedly compares the two values using `bgt/blt`, exactly like the starter code does, but instead of branching to a *different* section of code, subtracts the smaller value from the larger one and branches *back* to repeat the comparison.
3. Stops the loop when the two values become equal; this should give you the GCD value.
4. Prints the result with a clear label.

2.3.2 GCD of negative numbers

Write a MIPS32 program that:

1. Adapts the first code to allow the GCD computation of negative numbers.
2. Utilizes the `bltz` or `abs` function for this.

2.4 Requirements

- Use registers `$t0` and `$t1` to hold the two numbers being compared, just like the starter code uses `$t0` and `$t1`.
- Use a label (e.g., `gcd_loop:`) that the program branches back to.
- Use `beq` to detect when the loop should stop.
- End the program with the exit syscall (10).

2.5 Sample Run

```
Enter the first number: 48
Enter the second number: 18
```

```
The HCF (GCD) of the two numbers is: 6
```